

Аннотация

Работа посвящена разработке инструмента для фиксации и отслеживания изменений в структуре БД. Актуальность исследования связана с тем, что в большинстве проектов одновременно используются системы контроля версий (в том числе Git) и СУБД. При этом изменения в структуре базы данных часто выполняются постепенно: добавляются новые таблицы, меняются связи, индексы, ограничения, триггеры, представления и другие объекты. Если такие изменения не фиксировать и не отслеживать в понятном виде, то со временем становится сложно понять, что именно менялось, когда и по какой причине.

В проекте используются некоторые основные принципы Git для отслеживания изменений, стандартные средства взаимодействия с БД, а также простые элементы форматирования текста в стиле Markdown для ведения документации. Предполагается, что результат работы поможет упростить поддержку схемы базы данных, снизить число ошибок при изменениях и ускорить работу разработчиков и администраторов БД.

Содержание

Аннотация	1
Содержание	2
Введение	4
Перечень Сокращений и Обозначений	7
1 Теоретическая часть	8
1.1 Системы управления Базами Данных	9
1.1.1 Теория	10
1.1.2 Тенденции использования СУБД в мире	11
1.1.3 Обоснование выбора СУБД Firebird и Postgres	12
1.2 Системы контроля версий	13
1.2.1 Теория	14
1.2.2 Тенденции использования систем контроля версий	15
1.2.3 Обоснование выбора локальных систем контроля версий	16
1.3 Механизмы документирования систем контроля версий	17
1.3.1 Теория	17
1.3.2 Развития документирования продуктов	18
1.3.3 Обоснование выбора подхода к решению проблемы документирования	20
1.4 Вывод	21
2 Проектирование приложения для версионирования БД	23
2.1 Минимальные требования к программе	24
2.1.1 Общие требования	24
2.1.2 Требования к фронтенду	25
2.1.3 Требования к бэкенду	25

2.2 Сценарии взаимодействия с пользователем.....	27
2.3 Архитектура приложения.....	29
2.4 Включенные паттерны программирования.....	31
2.5 Особенности реализации локального хранилища	32
2.6 Особенности реализации чтения схем БД.....	33
2.7 Вывод.....	34
3 Разработка инструмента для версионирования баз данных	36
3.1 Используемые инструменты	36
3.2 Реализованные решения и соответствие требованиям.....	37
3.3 Тестирование реализованного продукта.....	40
3.4 Вывод.....	40
Источники	43

Введение

Актуальность темы исследования. В реалиях современной разработки программного обеспечения системы контроля версий, различные методики документирования стали стандартами. Практически во всех крупных современных информационных системах применяются различные системы хранения данных. С ходом процесса разработки хранилища данных, или же базы данных тоже имеют тенденцию развиваться, их структура растет и усложняется.

Цель исследования. Разработка сервиса документирования и версионирования схем базы данных.

Задачи исследования. Исследовать существующие подходы к управлению миграциями схем БД и автоматизации документирования.

Разработать архитектуру и алгоритмы работы сервиса, обеспечивающие автоматический захват состояния схемы БД и миграцию документации при каждом изменении.

Создать веб-интерфейс для визуализации документации и истории версий, позволяющий пользователям просматривать различия между версиями и текущую структуру БД.

Провести тестирование разработанного сервиса: выполнить модульное тестирование, проверить корректность отслеживания версий при различных сценариях изменения БД, оценить удобство использования и быстродействие сервиса.

Объект исследования. Процесс управления, поддержки и контроля изменений структуры реляционных баз данных при разработке и сопровождении программного обеспечения.

Предмет исследования. Методы, алгоритмы и технологии автоматизированного чтения, сохранения, сравнения и документирования данных об объектах схемы базы данных, таких как таблицы, связи, атрибуты, индексы и триггеры.

Теоретическая и практическая значимость работы. Теоретическая значимость заключается в систематизации подходов к версионированию структуры баз данных, анализе методов применения принципов систем контроля версий к объектам баз данных, а также в обосновании выбора архитектурных решений для работы с различными системами управления базами данных через унифицированные интерфейсы. Практическая значимость состоит в создании работоспособного программного продукта, упакованного в независимый образ, который может быть легко запущен локально или совместно с базой данных. Разработанный инструмент поможет существенно упростить поддержку схемы базы данных, снизить число возможных ошибок при внесении изменений, в сущности, и связи, а также ускорить повседневную работу разработчиков и администраторов баз данных. Результаты работы могут быть использованы в реальных проектах для улучшения процесса документирования баз данных и повышения прозрачности изменений в структуре хранения информации.

Методы исследования. В работе используются методы системного анализа предметной области, принципы объектно-ориентированного проектирования программного обеспечения, методы анализа структур данных и языка определения данных, а также подходы к проектированию веб-приложений и современные паттерны программирования. Экспериментальная проверка базируется на методах тестирования программных систем при взаимодействии с реальными экземплярами баз данных.

Структура и объем работы. Выпускная квалификационная работа состоит из введения, нескольких глав, заключения, списка использованных источников и приложений. Общий объем работы составляет 44 страниц.

Перечень Сокращений и Обозначений

СУБД – система управления базой данных

БД – база данных

UI – пользовательский интерфейс

SQL – язык структурированных запросов

DDL – язык определения данных

DML – язык манипулирования данными

1 Теоретическая часть

Любое коммерческое предприятие в процессе своей деятельности неизбежно стремится к оптимизации распределения ресурсов и снижению финансовых издержек. В разработки информационных технологий основная финансовая нагрузка разделяется на две категории: затраты на первоначальную разработку программного продукта и затраты на его последующее сопровождение и техническую поддержку [3].

По мере развития информационной системы и добавления новых бизнес-требований ее внутренняя сложность неизбежно возрастает. Увеличение масштабов проекта приводит к нелинейному росту расходов на разработку и поддержку [3]. Основная причина такого непропорционального увеличения затрат заключается в усложнении восприятия кодовой базы проекта программистом. Разработчикам требуется все больше рабочего времени исключительно на то, чтобы изучить существующую архитектуру, разобраться в обширной кодовой базе и понять логику взаимодействия различных компонентов системы перед внесением даже незначительных правок [1, 8].

Описанные трудности в равной степени относятся не только к программному коду приложения, но и к системам хранения данных, которые являются фундаментом практически любой современной IT-системы [6, 7]. База данных редко остается неизменной на протяжении всего жизненного цикла проекта. Схема данных постоянно эволюционирует: появляются новые сущности, добавляются таблицы, изменяются связи между ними, перерабатываются индексы и ограничения [2, 14]. Если данные процессы не контролировать, структура базы данных со временем становится запутанной, что многократно повышает риск внесения критических ошибок при добавлении нового функционала и увеличивает стоимость обслуживания всей системы [13].

Для того чтобы частично решить проблему понимания сложных систем и снизить риски, команды разработки активно применяют специализированные инженерные практики. Наиболее распространенными и эффективными подходами являются ведение подробной технической документации и обязательное использование систем контроля версий [1, 11]. Эти инструменты помогают сохранять историю изменений и предоставляют контекст для принимаемых технических решений, значительно облегчая процесс разработки [8].

Разрабатываемый в рамках данной работы программный продукт направлен на решение описанных проблем непосредственно в контексте систем управления базами данных. Приложение объединяет в себе подходы к документированию и принципы систем контроля версий, специализируя их исключительно для работы со структурами хранилищ данных [11, 14]. Инструмент предоставляет единую среду, в которой специалисты могут легко отследить историю изменения таблиц и связей между сущностями, ознакомиться с описанием каждого атрибута и сравнить различные состояния схемы [5, 9, 10]. Внедрение подобного автоматизированного инструмента призвано упростить повседневное взаимодействие с базами данных, снизить количество потенциальных ошибок при проектировании схемы и, как прямое следствие, существенно сократить временные и финансовые затраты на поддержку и развитие информационных систем [3, 7, 12].

1.1 Системы управления Базами Данных

Для хранения, организации и управления данными в современных компьютерных системах повсеместно применяются базы данных и системы управления базами данных [6, 7]. СУБД являются одним из самых главных элементов современной информационной инфраструктуры [2]. Они представляют собой надежное хранилище данных и обеспечивают строгие единые правила работы с информацией для всех пользователей и приложений

[3]. Подобные хранилища данных используются почти во всех сферах человеческой деятельности, включая образовательные учреждения, управление бизнес-процессами, банковское дело, а также решение различных научных и инженерных задач [7].

1.1.1 Теория

Система управления базами данных представляет собой комплекс программных средств, который управляет структурой базы данных и строго контролирует доступ к хранимой информации [6]. Данная система выступает незаменимым посредником между конечным пользователем или программным приложением и самой базой данных, которая на физическом уровне обычно является набором файлов на диске [3]. Система скрывает внутреннюю реализацию хранения данных от пользователя, предоставляя ему удобную логическую модель, состоящую из таблиц, представлений и запросов [2].

Главная задача СУБД заключается в том, чтобы позволить безопасно хранить огромное количество информации, обеспечивая при этом высокую скорость чтения и записи за счет оптимизации внутренних алгоритмов и строгой структуры сущностей и связей между ними [7].

В реляционных системах данные организованы в виде двумерных таблиц, где строки представляют собой конкретные записи, а столбцы описывают атрибуты сущностей [6]. Для обеспечения целостности информации используются первичные и внешние ключи, которые формируют надежные связи между различными таблицами [2]. Взаимодействие с данными осуществляется при помощи языка структурированных запросов, который позволяет эффективно извлекать, добавлять и изменять информацию [7]. Ключевой особенностью реляционных баз данных является поддержка транзакций, что гарантирует надежность выполнения операций даже в случае системных сбоев [3].

Нереляционные базы данных, также называемые NoSQL-системами, представляют собой альтернативный подход к организации и хранению информации [3]. В отличие от реляционных систем, такие базы данных не требуют строгого описания структуры таблиц перед началом работы с данными [2]. Вместо привычных таблиц, строк и столбцов нереляционные системы могут использовать различные модели организации данных: документно-ориентированные хранилища, графовые базы данных, а также колоночные хранилища[3]. Главным преимуществом нереляционных систем является высокая гибкость схемы данных, что позволяет легко изменять структуру хранимых записей без предварительных миграций [2]. Однако именно эта гибкость делает задачу строгого версионирования и отслеживания изменений схемы значительно сложнее и менее предсказуемой по сравнению с реляционными системами, где структура всегда четко определена и может быть точно зафиксирована в каждый момент времени [13].

1.1.2 Тенденции использования СУБД в мире

Современные тенденции в области управления данными показывают постоянный рост объемов хранимой информации и повышение требований к скорости ее обработки [3]. В мире активно развиваются облачные технологии, что приводит к переносу многих баз данных на удаленные серверы и использованию баз данных как услуги [12]. Однако, несмотря на появление множества альтернативных моделей хранения данных, реляционные системы управления базами данных продолжают занимать лидирующие позиции в корпоративном секторе [6, 7].

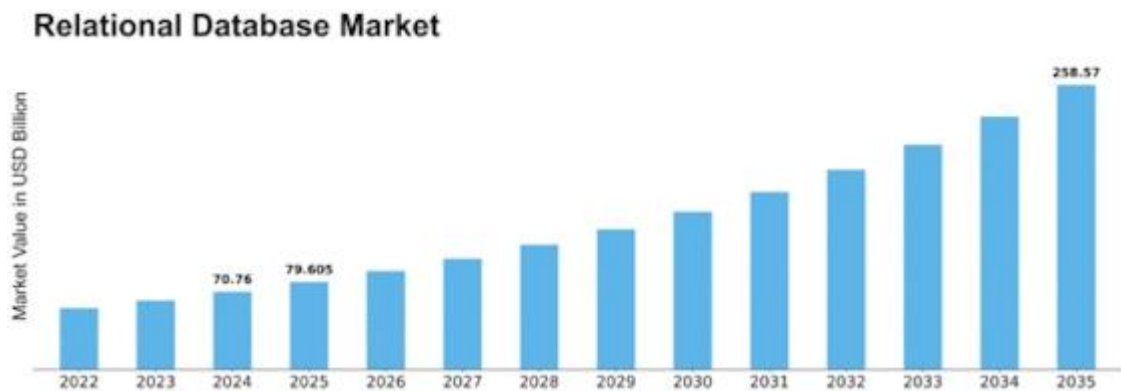


Рисунок 1.1 — График роста рынка систем управления баз данными.

Согласно исследованиям, глобальный рынок СУБД имеет стойкую тенденцию роста [12]. Благодаря тенденции бизнеса переносить как вычислительные мощности, так и хранилища на облачную инфраструктуру позиция СУБД укрепляется [12]. Также количество хранимой информации с каждым годом увеличивается, что стимулирует развитие систем хранения данных [3].

Это связано с тем, что большинство бизнес-приложений требуют строгой согласованности данных, четкой структуры сущностей и надежных связей между ними [6]. Еще одной важной тенденцией является интеграция процессов разработки баз данных с общими процессами создания программного обеспечения [14]. Разработчики все чаще стремятся применять подходы непрерывной интеграции и доставки к базам данных, что вызывает острую необходимость в инструментах для контроля версий структур баз данных [13]. Схема данных теперь рассматривается как часть программного кода, которая должна проверяться, тестироваться и развертываться автоматически [14].

1.1.3 Обоснование выбора СУБД Firebird и Postgres

В контексте разрабатываемого инструмента система управления базами данных является центральным объектом исследования и взаимодействия из-за их глобальной распространенности [6, 7].

Разрабатываемое приложение должно подключаться к таким системам для считывания текущей структуры таблиц, связей, индексов и триггеров [9, 10]. Без использования стандартизированных средств подключения к базам данных было бы невозможно автоматически получать актуальную информацию о схеме данных [2].

Выбор реляционных систем управления базами данных обусловлен тем, что они предоставляют строгие механизмы для контроля связей между сущностями и поддерживают транзакции [6]. Нереляционные базы данных обладают слишком гибкой и изменчивой схемой, что делает задачу строгого версионирования структуры менее предсказуемой [3].

В качестве конкретных вариантов для поддержки в проекте были выбраны системы PostgreSQL и Firebird [5, 9, 10].

Данные сайта StackOverflow показывают, что разработчики предпочитают использовать PostgreSQL [1]. Поэтому благодаря популярности данной СУБД и легкости её интеграции в проект было решено использовать её в качестве одной из поддерживаемых СУБД в проекте [5]. Другая используемая СУБД является Firebird, причина выбора этой СУБД кроется в необходимости повышения навыков с обращением данной СУБД из-за того, что она используется в профессиональной деятельности автора работы. Данная СУБД применяется в одном из важнейших продуктов компании, а также является основой для популярной российской СУБД РЕД База Данных [10].

1.2 Системы контроля версий

В процессе разработки программного обеспечения исходный код и сопутствующие файлы постоянно подвергаются изменениям [1]. Для того чтобы записывать эти изменения с течением времени и иметь возможность в любой момент вернуться к определенной версии, применяются системы контроля версий [8]. Такие системы необходимы для защиты проекта от

случайных потерь данных, отслеживания авторства изменений и понимания причин, по которым была внесена та или иная правка, также такие системы позволяют проводить анализ производительности разработчика [1]. Системы контроля версий позволяют разработчикам безопасно экспериментировать с новыми функциями, возвращать проект к исходному рабочему состоянию в случае возникновения критических ошибок, а также эффективно организовывать совместную работу множества специалистов над одним продуктом [8].

1.2.1 Теория

Система контроля версий представляет собой программный инструмент, который отслеживает и записывает изменения в наборе файлов с течением времени, позволяя в любой момент восстановить любое из сохраненных состояний [1]. Подобные системы предоставляют разработчикам возможность просматривать историю правок, сравнивать различные версии файлов, определять авторство конкретных изменений и возвращать проект к более раннему стабильному состоянию в случае обнаружения критических ошибок [8]. Таким образом, система контроля версий выступает полноценной средой для организации совместной командной работы и управления жизненным циклом программного продукта [1].

Основной принцип работы системы заключается в создании снимков состояния набора файлов в определенный момент времени. Когда пользователь фиксирует изменения, система сохраняет текущее состояние всех файлов и создает ссылку на этот снимок [8].

Существует несколько видов систем контроля версий, которые отличаются по своей архитектуре и модели работы с историей изменений [3]. Локальные системы контроля версий хранят всю историю изменений непосредственно на компьютере пользователя в специальной базе данных [1]. Такой подход прост

в реализации, однако он не предоставляет возможности организовать совместную работу нескольких разработчиков над одним проектом [3]. Централизованные системы контроля версий решают эту проблему путем размещения единого репозитория на выделенном сервере. Все участники команды подключаются к этому серверу и получают актуальную версию файлов, однако при недоступности сервера работа всей команды оказывается заблокированной [3]. Распределенные системы контроля версий устраняют этот недостаток, предоставляя каждому участнику команды полную копию всей истории изменений на его локальном компьютере [8]. Взаимодействие между участниками осуществляется путем обмена наборами изменений между локальными репозиториями. Такой подход обеспечивает максимальную надежность хранения истории, поскольку каждый участник разработки фактически является независимой резервной копией всего проекта [1]. Наиболее распространенным представителем распределенных систем контроля версий является Git, который на сегодняшний день стал фактическим стандартом в индустрии разработки программного обеспечения [8].

1.2.2 Тенденции использования систем контроля версий

В последние годы системы контроля версий стали абсолютным стандартом в индустрии разработки программного обеспечения [1]. Отсутствие системы версионирования в проекте в настоящее время считается серьезным нарушением инженерных практик [8]. Наблюдается тенденция к расширению применения этих систем за пределы обычного исходного кода [11].

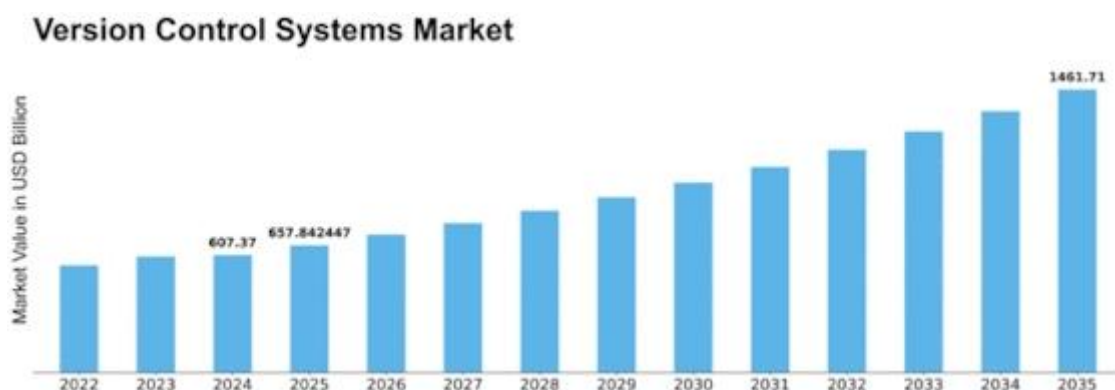


Рисунок 1.2 — График роста рынка систем контроля версий

Согласно исследованиям рынка тенденции использования систем контроля версии будут расти [12]. Это обусловлено тем, что бизнес переходит на облачные вычисления, что также способствуют использованию распределенных систем контроля версий [3]. Современные системы контроля версий позволяет автоматизировать или упростить часть процессов разработки и поддержки продуктов, что сильно снижает расходы бизнеса [1]. Наиболее популярными инструментами для автоматизации процессов разработки являются CI/CD и их аналоги [8].

Современные команды разработчиков применяют принципы версионирования для конфигурации серверов, инфраструктуры и документации [11]. Этот подход получил название инфраструктура как код [3]. Аналогично развивается направление база данных как код, при котором все изменения в схеме базы данных, добавление новых таблиц или связей описываются в виде текстовых файлов, которые затем сохраняются в системе контроля версий [14]. Это позволяет синхронизировать изменения в программном коде с изменениями в структуре базы данных, избегая ситуаций, когда новая версия приложения не может работать из-за устаревшей схемы данных [13].

1.2.3 Обоснование выбора локальных систем контроля версий

Принципы, заложенные в основу систем контроля версий, идеально подходят для решения задачи отслеживания изменений в структуре базы данных [14]. В разрабатываемом инструменте концепции создания локальных снимков состояния применяются к схеме базы данных [13]. Это позволяет сохранять историю развития таблиц и связей аналогично тому, как сохраняется история изменения текстовых файлов с исходным кодом [1].

Выбор архитектуры локального хранилища обусловлен тем, что локальное хранение снимков базы данных обеспечивает высокую скорость работы инструмента [3]. Приложению не нужно постоянно обращаться к

центральному серверу для получения истории изменений [8]. Все метаданные о структуре базы данных преобразуются в текстовый формат и сохраняются в специальной директории на диске компьютера пользователя [2]. Это делает инструмент простым в установке и использовании [11]. Кроме того, локальные файлы, созданные разрабатываемым инструментом, разработчики могут в дальнейшем самостоятельно добавлять в свои распределенные системы версионирования вместе с основным кодом проекта, обеспечивая единое пространство для всей истории разработки [8].

1.3 Механизмы документирования систем контроля версий

При разработке программного обеспечения и проектировании баз данных важную роль играет не только сам процесс создания структуры, но и ее подробное описание [11]. Документирование является неотъемлемой частью жизненного цикла любого проекта, поскольку фиксация технических решений позволяет командам эффективно взаимодействовать и поддерживать продукт на протяжении долгого времени [7]. В данном разделе рассматриваются теоретические основы документирования, прослеживается эволюция подходов к ведению технической документации в среде разработчиков, а также приводится обоснование выбора конкретного метода описания изменений для разрабатываемого инструмента версионирования [11].

1.3.1 Теория

Процесс документирования представляет собой целенаправленную фиксацию информации о структуре, алгоритмах и принципах работы разрабатываемой системы [11]. В контексте баз данных документация описывает сущности, их атрибуты, а также логические и физические связи между таблицами [6]. Качественное описание позволяет разработчикам, аналитикам и администраторам баз данных быстрее понимать логику хранения информации и механизмы работы продукта [7].

Существует несколько видов документации, среди которых выделяют пользовательскую, архитектурную и техническую [11]. Применительно к системам контроля версий и базам данных основное внимание уделяется технической документации, которая объясняет внутреннее устройство схемы данных [6]. Без надлежащего описания таблиц, индексов, триггеров и ограничений поддержка проекта со временем значительно усложняется, так как первоначальный замысел создателей забывается при смене состава команды [7]. Главной задачей технического описания выступает сохранение контекста принимаемых решений, то есть фиксация причин, по которым была создана конкретная сущность, изменена связь или добавлено новое поле [11].

1.3.2 Развития документирования продуктов

С течением времени подходы к документированию программного обеспечения претерпевали значительные изменения, отражая развитие самих технологий разработки [3]. На заре становления индустрии, до широкого распространения систем контроля версий, разработчики имели тенденцию фиксировать каждое действие программиста непосредственно в текстовых файлах исходного кода [1]. В комментариях подробно описывалось, кто именно изменял код, в какую дату были внесены правки и какие конкретно строки были добавлены или удалены [8]. Подобная информация быстро становилась избыточной, невероятно сильно перегружала файлы и усложняла чтение основного алгоритма [3].

С развитием систем контроля версий, таких как Git, и появлением современных интегрированных сред разработки этот подход быстро потерял свою актуальность [1]. Системы контроля версий взяли на себя задачу сохранения истории изменений, авторства и дат, освободив исходный текст от лишних метаданных [8]. Кроме того, эволюция высокоуровневых языков программирования и инструментов статического анализа привела к тому, что синтаксис стал более понятным и читаемым [3]. Современный грамотно

написанный код сам по себе подсказывает, какие действия он выполняет, поэтому парадигма использования комментариев изменилась [8]. В современной практике комментарии внутри кода принято оставлять только для объяснения неочевидного поведения программы, описания сложной бизнес-логики или обоснования выбранного технического решения, отвечая на вопрос почему это сделано именно так, а не что именно здесь написано [11].

Одновременно с этим документация, существующая в отрыве от исходного кода, также прошла длительный путь развития [11]. Постепенно сформировалось понимание, что одного только документирования строчек кода недостаточно для успешной поддержки и развития крупных проектов [3]. Появились различные стандартизированные способы описания архитектуры систем, моделей данных, сценариев поведения пользователя и взаимодействия компонентов между собой [7]. Значительное распространение получила практика вынесения инструкций по сборке, настройке окружения и первому запуску системы в отдельные специализированные файлы, которые хранятся в корне проекта [11].

Эволюция подходов к документированию закономерно затронула и сферу баз данных [6]. Долгое время структура баз данных описывалась исключительно в тяжеловесных текстовых документах или таблицах, которые создавались на этапе проектирования и практически не обновлялись после запуска системы в эксплуатацию [7]. Это приводило к тому, что при добавлении новых сущностей, изменении связей или удалении индексов реальная структура базы данных начинала сильно отличаться от заявленной в документации [6].

В современных реалиях к базам данных начинают применять те же принципы, что и к программному коду [14]. Возникает необходимость не просто фиксировать текущее состояние таблиц и связей, но и вести историю их изменений с подробными пояснениями [13].

1.3.3 Обоснование выбора подхода к решению проблемы документирования

В рамках разрабатываемого инструмента для отслеживания изменений в структуре баз данных возникает острая необходимость прикреплять различные пояснения и метаданные к снимкам схемы [14]. Пользователям системы нужно доступно объяснять, зачем была создана та или иная таблица, для чего добавлена новая связь между сущностями или почему в целях оптимизации был удален определенный индекс [13].

Для реализации этой задачи был выбран подход использования облегченного языка разметки Markdown [11]. Данный формат позволяет пользователям писать структурированные комментарии к изменениям связей или сущностей в привычном для них текстовом виде [11]. Впоследствии разрабатываемый инструмент может корректно распознавать эти метки и преобразовывать текст в легко читаемый вид в пользовательском интерфейсе [11]. Аналогами данной технологии могут выступать классические языки гипертекстовой разметки, однако они перегружены служебными символами и тегами, что делает исходный текст крайне трудным для восприятия без использования специального визуального редактора [11].

Выбранный формат является негласным отраслевым стандартом для документирования программных проектов и баз данных [11]. Он поддерживается абсолютным большинством современных платформ для совместной разработки и размещения кода [1]. Интеграция именно этого подхода в инструмент версионирования является наиболее логичным и обоснованным решением, так как техническим специалистам не придется переучиваться или использовать сторонние программы для написания пояснений к изменениям базы данных [11]. Актуальность и высокая востребованность данного формата среди разработчиков регулярно

подтверждается глобальными исследованиями ИТ-индустрии, в том числе масштабным опросом профильного портала StackOverFlow за 2025 год [1].

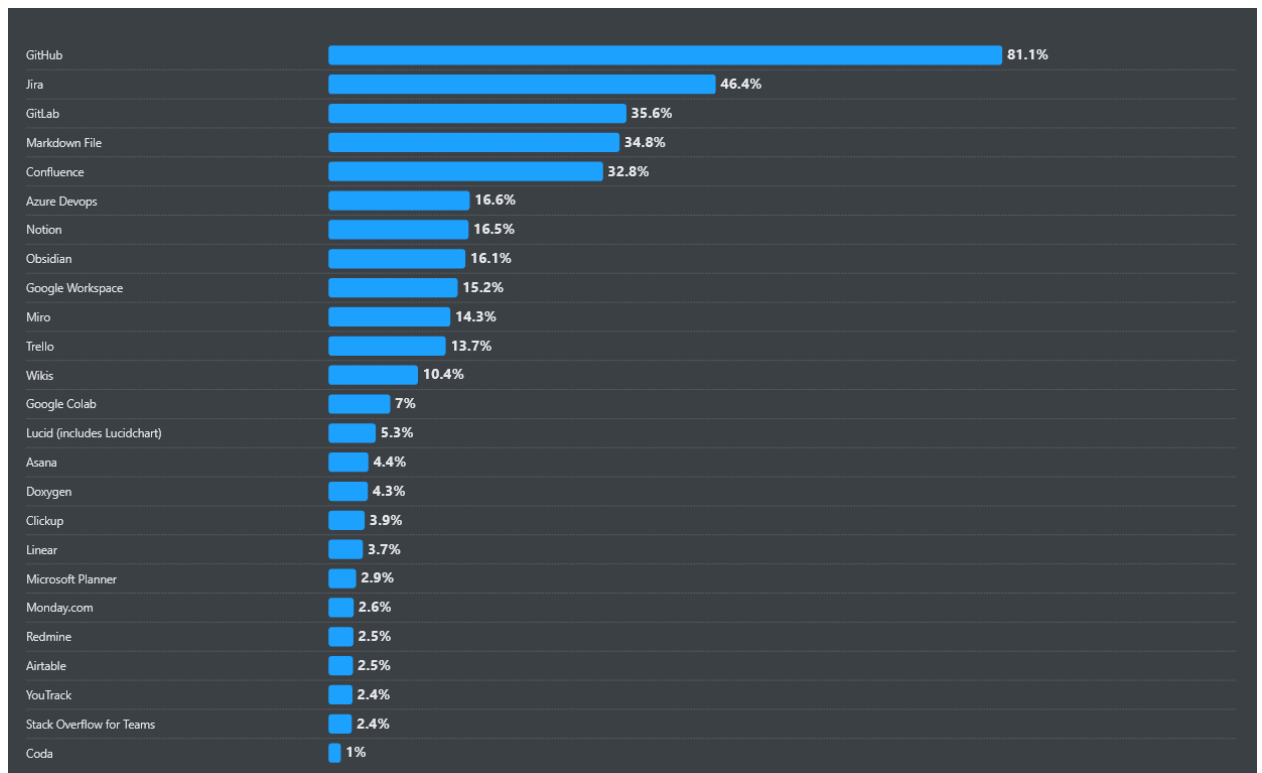


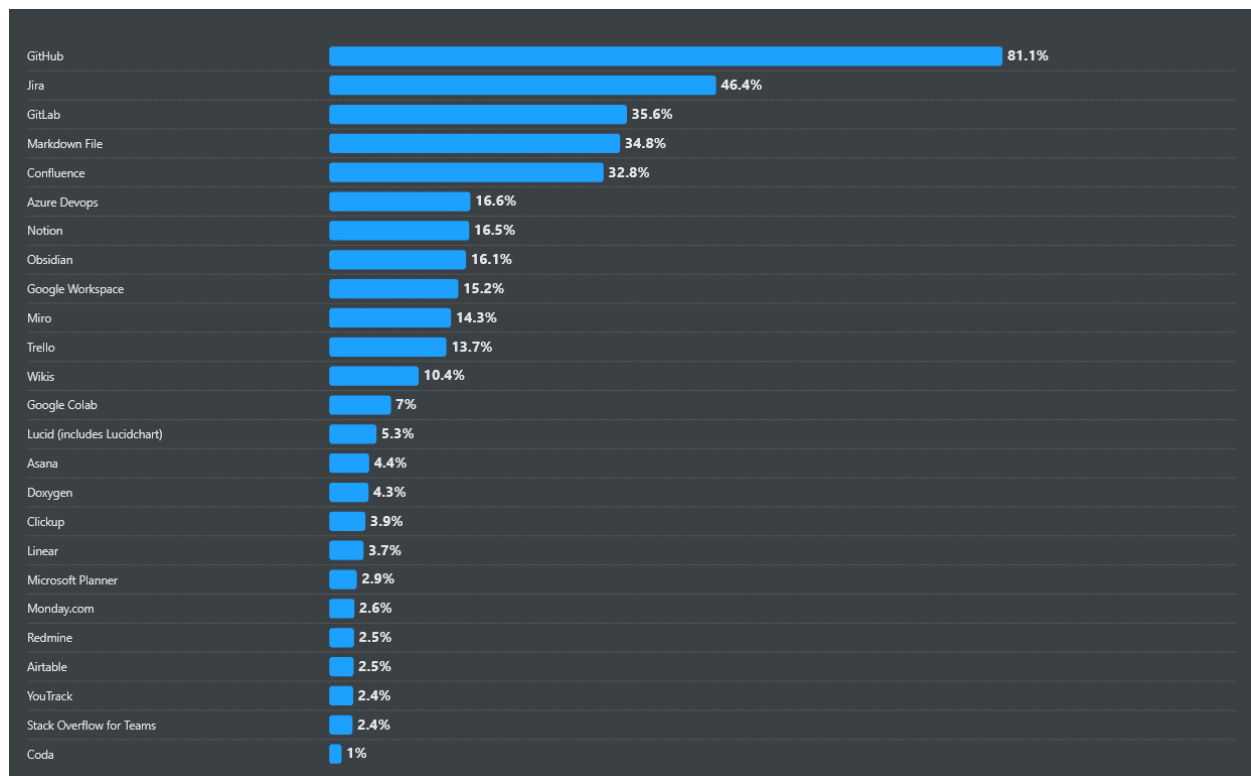
Рисунок 1.3 — График, отображающий источники документации в проектах.

1.4 Вывод

В ходе первой главы был проведен теоретический анализ предметной области, связанной с разработкой инструмента для версионирования структуры баз данных. Было установлено, что изменения в структуре данных, такие как добавление новых сущностей, изменение связей между ними и модификация ограничений, являются неизбежным процессом при развитии программного обеспечения. Отсутствие контроля над этими изменениями приводит к ошибкам и усложняет поддержку систем. В качестве основного объекта исследования были рассмотрены системы управления базами данных, и был сделан обоснованный выбор в пользу реляционных моделей на примерах PostgreSQL и Firebird, так как они обладают строгой структурой и механизмами контроля целостности.

Также были рассмотрены концепции систем контроля версий. Было принято решение применить принципы локального версионирования к схемам баз данных, что позволит сохранять историю структуры в виде локальных снимков. Для документирования изменений и описания связей выбран подход с использованием языка разметки Markdown, что упростит пользовательский опыт и обеспечит совместимость с современными практиками разработки. Сформированная теоретическая и методологическая база позволяет перейти к этапу непосредственного проектирования архитектуры программного продукта.

2 Проектирование приложения для версионирования БД



Процесс создания программного продукта начинается не с непосредственного написания исходного кода, а с тщательного предварительного проектирования. На этом этапе закладывается фундамент будущей системы: разрабатывается общая архитектура, определяются ключевые особенности функционирования, выстраиваются логические взаимосвязи между отдельными компонентами и модулями. Грамотный подход к проектированию позволяет избежать множества критических ошибок на этапе реализации, снижает затраты на последующую поддержку и гарантирует, что итоговый продукт может легко масштабироваться при добавлении нового функционала.

В данной главе будут рассмотрены минимальные требования к разрабатываемому продукту, описаны основные сценарии взаимодействия системы с конечным пользователем, а также разобраны архитектурные решения и паттерны программирования, которые легли в основу создаваемого инструмента для отслеживания и документирования структуры баз данных.

2.1 Минимальные требования к программе

Для того чтобы итоговый программный продукт в полной мере соответствовал целям дипломной работы и соответствовал поставленным целям и теме работы, необходимо четко определить ряд минимальных требований. Данные требования выступают в роли базовых критериев успешности разработки, задают границы функциональности системы и определяют направление дальнейшего проектирования интерфейса и внутренней логики.

Формирование перечня требований базируется на проведенном ранее анализе предметной области и стандартных практиках разработки.

2.1.1 Общие требования

Продукт должен корректно работать со схемой базы данных: подключаться к экземплярам БД, считывать структуру, интерпретировать её без потерь и отображать пользователю полный и актуальный набор поддерживаемых элементов. Представление схемы должно формироваться в нескольких заранее определенных форматах и оставаться согласованным между ними, чтобы один и тот же объект схемы одинаково распознавался и отображался независимо от выбранного варианта вывода. Инструмент должен поддерживать работу с несколькими СУБД и уметь одновременно обслуживать несколько подключений, при этом допускается, что добавленные экземпляры принадлежат к разным типам СУБД. Для каждого подключенного экземпляра продукт обязан корректно обрабатывать различия диалектов и системных особенностей, обеспечивая сопоставимое и предсказуемое отображение структуры, а также стабильность результатов при повторных чтениях.

Информация о схеме для всех добавленных экземпляров должна периодически и автоматически обновляться без участия пользователя, чтобы интерфейс и результаты экспорта отражали текущее состояние БД.

Обновление должно выполняться безопасно и ненавязчиво, не нарушая работоспособность целевой БД и не создавая заметной нагрузки, а при временной недоступности подключения продукт должен сохранять последнее успешное состояние и корректно восстанавливаться после появления доступа. Инструмент должен надёжно обрабатывать ошибки соединения и чтения метаданных, предоставляя понятные сообщения о причинах проблем и позволяя продолжать работу с остальными подключениями.

2.1.2 Требования к фронтенду

Интерфейс должен быть выполнен в минималистичном стиле и при этом оставаться достаточным для повседневных сценариев работы со схемой и её версиями. Пользователь должен легко ориентироваться в приложении без дополнительного обучения, быстро находить нужные экземпляры БД и элементы схемы, понимать текущее состояние обновления и различать, какие данные актуальны, а какие относятся к ранее сохранённому снимку. Взаимодействие должно быть последовательным и предсказуемым, с аккуратной терминологией и визуальной иерархией, чтобы продукт одинаково удобно воспринимался как при просмотре структуры, так и при выводе схемы в выбранном формате.

2.1.3 Требования к бэкенду

Бэкэнд приложения должен обеспечивать стабильную работу основного функционала, включая управление подключениями к базам данных, сбор метаданных и сохранение версий схем. Бэкэнд должен надёжно обрабатывать запросы от пользовательского интерфейса. Также бэкэнд должен реализовывать механизмы фонового обновления данных, чтобы периодически сканировать подключенные базы данных без блокировки основного потока выполнения приложения. Все внутренние ошибки, возникающие при чтении схем или записи в хранилище, должны корректно обрабатываться и

логироваться для последующего анализа, не приводя к аварийному завершению работы всего инструмента.

2.2 Сценарии взаимодействия с пользователем

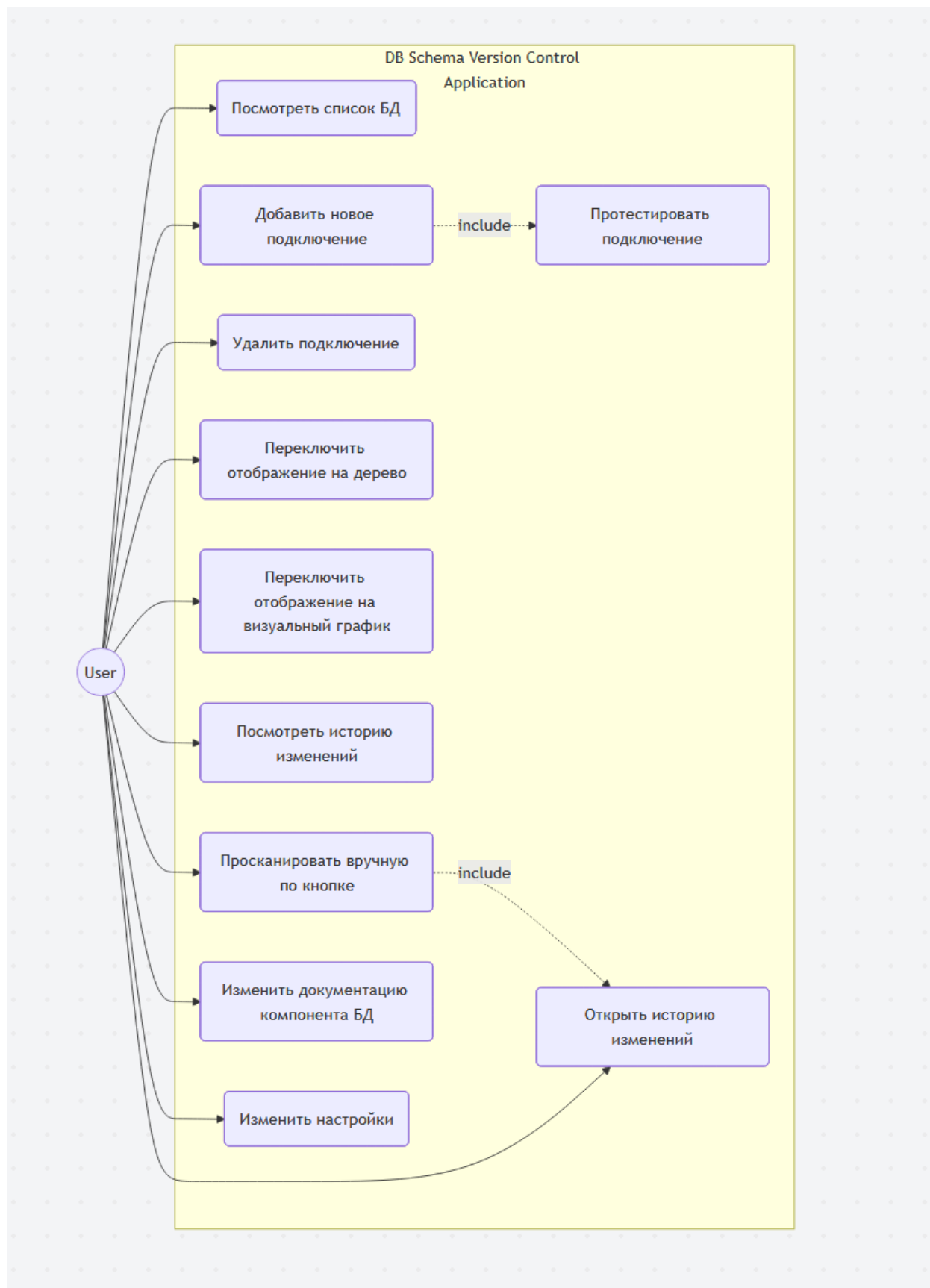


Рисунок 1.1 — Схема сценариев взаимодействия.

Взаимодействие пользователя с системой описывается через набор заранее определенных сценариев. Основным участником является системный пользователь.

Просмотр списка баз данных предполагает получение перечня всех зарегистрированных подключений. При открытии главной страницы система считывает конфигурации и выводит список баз данных, а при их отсутствии предлагает добавить новое подключение.

Добавление конфигурации базы данных позволяет зарегистрировать новое подключение для мониторинга. Пользователь вводит параметры сервера, логин и пароль. Этот процесс включает обязательную проверку подключения, при успехе которой новая конфигурация сохраняется в системе. Проверка подключения необходима для подтверждения корректности введенных параметров. Система пытается установить соединение через соответствующий модуль и информирует пользователя о результате.

Удаление конфигурации базы данных позволяет полностью очистить информацию о подключении и всю накопленную историю версий схемы, освобождая дисковое пространство.

Просмотр схемы в виде дерева дает возможность изучить структуру конкретной версии базы данных в иерархическом представлении. Система строит логическое дерево компонентов, включая таблицы, столбцы и индексы, для удобной навигации пользователя.

Визуальный просмотр схемы предоставляет наглядное отображение таблиц и связей между ними в формате графической схемы отношений сущностей.

Просмотр истории изменений схемы позволяет получить хронологический список версий. Система формирует отчеты об изменениях между соседними снимками, показывая количество добавленных, измененных или удаленных объектов базы данных.

Ручное сканирование базы данных инициирует немедленное создание новой версии схемы по текущему состоянию. Система подключается к

серверу, считывает структуру и сохраняет новую версию, если были обнаружены отличия от предыдущего состояния.

Просмотр детализации версии используется для изучения подробного состава сохраненного снимка схемы и списка конкретных изменений относительно прошлой версии.

Обновление документации к компоненту схемы позволяет пользователю добавить текстовое описание для любой таблицы или столбца. Этот текст сохраняется и автоматически переносится во все последующие версии схемы базы данных.

Изменение настроек приложения дает возможность управлять параметрами автоматического сканирования, внешним видом и языком пользовательского интерфейса.

2.3 Архитектура приложения

В разрабатываемом приложении применяется принцип луковой архитектуры, который обеспечивает высокую степень независимости компонентов системы и значительно упрощает дальнейшую поддержку и развитие проекта. Основой данного подхода является логическое разделение программного кода на несколько независимых уровней. Каждый из этих уровней выполняет свою строго определенную задачу и взаимодействует с остальными частями приложения исключительно через заранее установленные абстракции и интерфейсы, что позволяет минимизировать связность программных модулей.

Одним из важнейших компонентов системы является слой представления, который берет на себя ответственность за взаимодействие с конечным пользователем. На этом архитектурном уровне определяются основные маршруты приложения, а также формируются представления и подключаются соответствующие визуальные шаблоны. Данный слой критически необходим для корректной работы пользовательского интерфейса,

обеспечивая прием запросов от клиента, передачу их во внутренние механизмы системы и последующий возврат обработанного результата в наглядном виде. Важным правилом является то, что слой представления не содержит в себе сложных вычислительных алгоритмов или бизнес-логики, что позволяет при необходимости изменять визуальную часть приложения без какого-либо влияния на внутренние процессы.

Центральное место во всей архитектуре занимает слой ядра. В нем сосредоточена основная бизнес-логика приложения, включая правила обработки информации, валидацию поступающих данных и управление сущностями предметной области, такими как структуры таблиц, связи между сущностями, индексы и триггеры баз данных. Слой ядра полностью изолирован от деталей реализации любых внешних сервисов, баз данных или пользовательских интерфейсов. Это гарантирует его максимальную независимость и позволяет легко проводить автоматизированное тестирование. Вся работа с информацией на этом уровне происходит через абстрактные модели, благодаря чему ядро остается стабильным даже при существенных изменениях во внешних модулях системы.

Слой адаптеров выполняет роль связующего звена между изолированным ядром приложения и внешним миром. Этот уровень реализует конкретные механизмы взаимодействия с внешними источниками данных, такими как различные системы управления базами данных, файловые системы или сторонние программные интерфейсы. Адаптеры получают абстрактные вызовы от слоя ядра и преобразуют их в специфические команды и запросы, которые понятны целевой системе. Использование такого подхода совместно с контейнером внедрения зависимостей позволяет разработчикам легко подключать новые типы баз данных или изменять технические детали работы с существующими инфраструктурными компонентами, абсолютно не затрагивая при этом центральную бизнес-логику и механизмы отображения информации пользователю.

2.4 Включенные паттерны программирования

Для обеспечения высокого качества программного кода и упрощения дальнейшей поддержки в приложении применяются современные паттерны проектирования. Одним из ключевых используемых решений является паттерн Адаптер. Данный паттерн позволяет отделить общую логику работы с базой данных от специфичных деталей реализации конкретного подключения. Благодаря Адаптеру приложение может взаимодействовать с несколькими различными системами управления базами данных, каждая из которых имеет свои особенности подключения и форматы метаданных. Адаптер скрывает эти различия и предоставляет унифицированный интерфейс для других компонентов системы.

Дополнительно в архитектуру внедрен паттерн контейнера внедрения зависимостей. Использование этого паттерна освобождает классы бизнес-логики от необходимости самостоятельно создавать объекты, от которых они зависят. Контейнер берет на себя ответственность за создание и предоставление нужных реализаций, например, подставляя подходящий адаптер базы данных в зависимости от настроек пользователя. Это делает компоненты системы более независимыми друг от друга, упрощает процесс тестирования и повышает общую гибкость разрабатываемого инструмента.

Также в архитектуре, активно применяется паттерн Репозиторий. Он выступает в роли промежуточного слоя между бизнес-логикой и логикой доступа к данным, позволяя полностью абстрагироваться от конкретного способа сохранения снимков схем в локальной базе приложения. Для создания сложных объектов, описывающих связи и сущности схемы данных, используется паттерн Фабрика. Этот подход централизует логику генерации внутренних моделей на основе сырых метаданных, полученных из внешних систем управления базами данных. Все эти подходы делают компоненты системы более независимыми друг от друга, упрощают процесс тестирования и повышают общую гибкость разрабатываемого инструмента.

2.5 Особенности реализации локального хранилища

Хранение данных в приложении реализовано с учетом необходимости надежной фиксации изменений и быстрого доступа к историческим снимкам схем баз данных. В основе локального хранилища лежит принцип разделения конфигурационных данных и самих сохраненных версий схем. Для управления сохраненной информацией применяется подход, который изолирует логику записи и чтения от остальной части системы, предоставляя простой интерфейс для работы с историей изменений.

Алгоритм сохранения новой версии схемы базируется на принципе создания полных снимков состояния. При каждом успешном сканировании подключенной базы данных формируется структурированный документ, содержащий всю необходимую информацию о таблицах, связях, ключах и индексах. Для оптимизации процесса поиска отличий между версиями применяется алгоритм пошагового сравнения деревьев объектов. При запросе истории изменений система загружает два смежных снимка и последовательно проходит по их узлам, выявляя добавленные, удаленные или модифицированные элементы структуры.

Для хранения метаданных и настроек системы используется локальная реляционная база данных, что гарантирует целостность операций и защиту от потери информации при системных сбоях. Сами исторические снимки схем преобразуются в текстовые форматы и сохраняются в дисковой подсистеме. Такое разделение снижает нагрузку на основную базу данных приложения и существенно упрощает процесс резервного копирования и восстановления исторической информации. Все операции взаимодействия с хранилищем строго изолированы, что соответствует общим принципам чистой архитектуры, заложенным в проекте.

2.6 Особенности реализации чтения схем БД

Процесс чтения структуры целевой базы данных строится на механизмах извлечения метаданных, предоставляемых самими системами управления базами данных. Теоретическая реализация этого процесса основана на последовательном обращении к системным каталогам и информационным схемам, которые хранят детальные сведения обо всех зарегистрированных объектах базы данных.

Для обеспечения независимости инструмента от конкретного типа применяемой базы данных используется система адаптеров. При инициализации процесса сканирования система выбирает нужный программный интерфейс, соответствующий типу текущего подключения. Алгоритм чтения выполняется строго поэтапно. На первом этапе выполняются стандартизированные запросы для получения общего списка всех существующих таблиц. На втором этапе для каждой найденной таблицы запрашиваются сведения о столбцах, применяемых типах данных, наложенных ограничениях и существующих индексах. На третьем, заключительном этапе, извлекаются данные о внешних ключах, которые формируют связи между таблицами, а также собирается информация о системных триггерах.

Полученные из системных таблиц необработанные данные передаются в специальные компоненты системы, которые преобразуют их во внутренние информационные модели приложения. Этот процесс нормализации скрывает различные особенности языков запросов и форматы ответов разных поставщиков баз данных. Таким образом, слой основной логики приложения всегда работает с единым и понятным представлением таблиц и связей, независимо от источника информации. Если в процессе сканирования возникают ошибки доступа или обрывы сетевого соединения, алгоритм предусматривает механизмы безопасного прерывания операции с

информированием пользователя и сохранением последнего стабильного состояния приложения.

2.7 Вывод

В рамках второй главы было проведено комплексное проектирование приложения для отслеживания изменений и документирования структуры баз данных. На основе проведенного ранее теоретического анализа предметной области были сформулированы четкие минимальные требования к разрабатываемому инструменту. Описание сценариев взаимодействия с пользователем позволило определить полный жизненный цикл работы с программой, начиная от первичной настройки подключения и заканчивая просмотром визуальных схем отношений сущностей и написанием комментариев к отдельным столбцам или таблицам.

Особое внимание при проектировании было уделено выбору архитектурных решений. Использование принципов луковой архитектуры позволило логически разделить приложение на независимые слои представления, центрального ядра и внешних адаптеров. Применение классических паттернов программирования, среди которых выделяются Адаптер, Фабрика, Репозиторий и контейнер внедрения зависимостей, обеспечило высокую гибкость и модульность разрабатываемого продукта. Эти проектные решения дают возможность легко расширять функционал приложения в будущем, позволяя добавлять поддержку новых баз данных без необходимости масштабного переписывания существующего программного кода.

Также в главе были детально рассмотрены технические особенности реализации локального хранилища данных и алгоритмы чтения структуры баз данных. Выбранный подход к хранению информации, предполагающий разделение конфигурационных настроек и самих исторических снимков схем в текстовом формате, обеспечивает общую надежность работы инструмента и

существенно снижает нагрузку на базу данных самого приложения. Описанный механизм поэтапного сканирования системных каталогов позволяет максимально точно и безопасно извлекать метаданные о таблицах, типах данных, ограничениях, триггерах и внешних ключах. Последующая нормализация полученных сырых данных гарантирует, что внутри приложения вся собранная информация всегда будет представлена в едином универсальном виде, который удобно анализировать и сравнивать для выявления отличий между сохраненными снимками.

Проведенное проектирование заложило прочный и надежный фундамент для последующей программной реализации инструмента версионирования.

3 Разработка инструмента для версионирования баз данных

3.1 Используемые инструменты

Для реализации приложения был подобран набор технологий, каждая из которых отвечает за строго определенную часть системы. Выбор каждого инструмента обоснован его соответствием архитектурным требованиям, описанным во второй главе.

Основным языком программирования является Python. Данный язык имеет большую аудиторию, что позволяет легко с ним работать за счет большого количества материала в открытом доступе. Так же большое количество библиотек упрощает разработку, а простой и понятный синтаксис позволяет разобраться в любом коде.

В качестве веб-фреймворка используется Django. Данный фреймворк предоставляет готовые механизмы для работы с HTTP-запросами, маршрутизации URL-адресов, формирования HTML-шаблонов и обработки форм. Встроенная система шаблонов Django позволяет отделить логику представления от бизнес-логики. Кроме того, Django включает встроенный веб-сервер для разработки.

Для работы с целевыми базами данных применяются специализированные драйверы. Взаимодействие с PostgreSQL осуществляется через библиотеку `psycopg2`, которая является наиболее распространенным адаптером для данной СУБД на языке Python. Подключение к базам данных Firebird реализовано через библиотеку `fdb`, которая обеспечивает взаимодействие с данной СУБД посредством её клиентских библиотек.

Управление зависимостями между компонентами системы выполняется с помощью библиотеки `Dishka`. Данная библиотека реализует паттерн

контейнера внедрения зависимостей и позволяет описывать способ создания объектов через провайдеры.

Снимки схем баз данных сохраняются в виде файлов в формате JSON. Данный формат является текстовым, хорошо читается человеком и легко обрабатывается средствами языка Python без использования дополнительных зависимостей.

Для формирования пользовательского интерфейса на стороне клиента применяется CSS-фреймворк Bootstrap совместно с набором иконок Bootstrap Icons. Данный набор инструментов позволяет создавать интерфейс без необходимости написания большого количества собственных стилей.

Для развертывания приложения используется Docker и Docker Compose. Контейнеризация обеспечивает воспроизводимость запуска на любой операционной системе и изолирует зависимости приложения от окружения хост-машины. Управление зависимостями Python внутри контейнера осуществляется с помощью менеджера пакетов uv, который отличается значительно более высокой скоростью установки пакетов по сравнению со стандартным инструментом pip.

Тестирование проекта проводится с использованием фреймворка pytest. Данный инструмент обладает удобным синтаксисом для написания тестовых функций и классов, широкими возможностями для параметризации тестов и поддержкой расширений.

3.2 Реализованные решения и соответствие требованиям

В данном разделе рассматривается соответствие разработанного продукта минимальным требованиям, сформулированным во второй главе дипломной работы.

Требование о корректной работе со схемой базы данных реализовано через механизм поэтапного сканирования системных каталогов. Для каждой из поддерживаемых СУБД разработан отдельный набор SQL-запросов к

системным таблицам. Для PostgreSQL запросы обращаются к представлениям схемы `information_schema`, а для Firebird используются системные таблицы `RDB$`. Алгоритм последовательно извлекает список таблиц, для каждой таблицы получает перечень столбцов с их типами данных и ограничениями, после чего собирает информацию об индексах, внешних ключах, уникальных ограничениях и триггерах. Полученные сырые данные нормализуются в единые внутренние сущности приложения, взятые из слоя доменов. Это гарантирует, что слой бизнес-логики всегда работает с одинаковым представлением объектов схемы независимо от типа подключенной СУБД.

Требование поддержки нескольких подключений одновременно обеспечивается тем, что каждая конфигурация базы данных хранится как отдельная запись в файле конфигурации. Каждому подключению присваивается уникальный идентификатор, под которым в отдельной директории хранятся все исторические снимки схемы данной базы. Пользователь может добавить любое количество подключений к базам данных различных типов, и все они будут независимо отображаться в главном списке приложения.

Требование автоматического обновления данных реализовано через страницу настроек, где пользователь может указать интервал сканирования в секундах и включить режим автоматического сканирования. При включенном автосканировании приложение периодически подключается ко всем зарегистрированным базам данных, считывает текущую структуру и сохраняет новую версию только в случае обнаружения отличий от последнего сохраненного снимка. Таким образом, история версий не засоряется идентичными записями.

Требование надежной обработки ошибок соединения выполнено на уровне всех операций подключения к базам данных. При неудачной попытке соединения адаптер генерирует исключение, которое перехватывается на уровне представления и преобразуется в понятное пользователю сообщение

об ошибке. При этом работа с остальными подключениями продолжается в штатном режиме.

Требование к функционалу версионирования выполнено в полном объеме. Сервий версий создает объект версии с уникальным идентификатором, датой создания и полным снимком схемы. При создании новой версии сервис автоматически переносит все ранее добавленные пользователем описания из предыдущей версии во все компоненты, которые не были удалены из схемы. Это позволяет сохранять накопленную документацию при каждом обновлении структуры базы данных.

Требование к отображению изменений выполнено через сервис поиска изменений. При запросе истории версий система загружает два соседних снимка и последовательно сравнивает их деревья объектов, формируя отчет об изменениях, в котором перечислены все добавленные, удаленные и модифицированные таблицы, столбцы, индексы, триггеры и внешние ключи. В интерфейсе пользователю отображается сводная информация о количестве изменений каждого типа, а также возможность перейти к детальному просмотру конкретной версии.

Требование к документированию реализовано через механизм хранения описаний компонентов схемы в виде отдельных записей внутри каждого объекта схемы версий. Пользователь может добавить текстовое описание в формате Markdown для любой таблицы, столбца, индекса, триггера или внешнего ключа. В пользовательском интерфейсе этот текст отображается в преобразованном виде с помощью библиотеки `marked.js`.

Требование к визуализации схемы выполнено через страницу визуального просмотра. На данной странице для каждой таблицы формируется блок с перечнем столбцов, а между таблицами, связанными внешними ключами, отрисовываются стрелки на SVG-канвасе. Пользователь может перемещать блоки таблиц по экрану и масштабировать диаграмму.

3.3 Тестирование реализованного продукта

Для проверки корректности работы основных компонентов приложения была разработана система модульных тестов. Тестирование охватывает ключевые сервисы и сущности доменного слоя, поскольку именно они содержат критическую бизнес-логику, от которой зависит надежность всего инструмента.

Все тесты расположены в директории тестов и разделены на файлы по принципу принадлежности к конкретному компоненту системы. Каждый тестовый метод следует единому шаблону, при котором сначала подготавливаются входные данные, затем вызывается тестируемый метод и в конце проверяется соответствие результата ожидаемому значению. Такой подход делает тесты легко читаемыми и изолированными друг от друга.

Для корректной работы некоторых тестов применяются заглушки. Применение данного паттерна позволяет оторвать тестирование конкретного компонента, от других связанных с ним через зависимости.

Все разработанные тесты успешно проходят проверку. Итоговые результаты запуска тестового набора подтверждают корректность реализации основных алгоритмов: обнаружения изменений в схеме базы данных, создания и сериализации версий, переноса пользовательской документации между версиями и управления конфигурацией приложения.

3.4 Вывод

В ходе третьей главы была осуществлена практическая реализация инструмента для версионирования и документирования структуры баз данных, спроектированного во второй главе. Для достижения поставленных целей был сформирован технологический стек, включающий Python 3.12, веб-фреймворк Django, библиотеку Dishka для управления зависимостями,

драйверы psycorg2 и fdb для работы с целевыми базами данных, а также Docker для контейнеризации и воспроизводимого развертывания.

Реализованный продукт в полной мере отвечает минимальным требованиям, сформулированным на этапе проектирования. Механизм поэтапного сканирования системных каталогов обеспечивает корректное и полное считывание структуры таблиц, столбцов, индексов, внешних ключей и триггеров как для PostgreSQL, так и для Firebird. Система адаптеров скрывает различия между диалектами этих СУБД и предоставляет бизнес-логике унифицированное представление схемы данных. Алгоритм сравнения версий позволяет точно выявлять добавленные, удаленные и модифицированные элементы структуры между двумя соседними снимками. Механизм наследования документации гарантирует, что пользовательские описания таблиц и столбцов автоматически переносятся в каждую новую версию схемы, не требуя повторного ввода.

Проведенное модульное тестирование охватило наиболее критичные части приложения: сущности доменного слоя, сервис обнаружения изменений, сервис преобразования версий и сервис управления конфигурацией. Все тесты успешно прошли проверку, что подтверждает правильность реализации заложенной логики. Использование объектов-заглушек при тестировании сервисов, зависящих от файловой системы, позволило обеспечить изоляцию тестов и их стабильное выполнение в любом окружении.

Таким образом, разработанный инструмент решает поставленную в начале работы задачу: он предоставляет специалистам единую среду для отслеживания истории изменений структуры базы данных, ведения технической документации и визуального просмотра схемы. Архитектурные решения, принятые на этапе проектирования, доказали свою практическую ценность: луковая архитектура и паттерн адаптера позволили легко реализовать поддержку двух различных СУБД без дублирования бизнес-логики, а применение контейнера внедрения зависимостей существенно

упростило написание изолированных тестов для каждого компонента системы.

Источники

1. Чакон С., Штрауб Б. Git для профессионального программиста. – Санкт-Петербург: Питер, 2016. – 496 с.
2. Мамедли М. Системы управления данными. – Москва: ДМК Пресс, 2020. – 480 с.
3. Клеппман М. Высоконагруженные приложения. – Санкт-Петербург: Питер, 2018. – 640 с.
4. Официальная документация Firebird // Firebird Foundation. – URL: <https://firebirdsql.org/en/documentation/> (дата обращения: 18.04.2026).
5. Официальная документация PostgreSQL // PostgreSQL Global Development Group. – URL: <https://www.postgresql.org/docs/> (дата обращения: 18.04.2026).
6. Дейт К.Дж. Введение в системы баз данных. – 8-е изд. – Москва: Вильямс, 2005. – 1328 с.
7. Коннолли Т., Бегг К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика. – 6-е изд. – Москва: Вильямс, 2017. – 1440 с.
8. Лубанкович М. PostgreSQL: основы, настройка, администрирование. – Москва: ДМК Пресс, 2020. – 480 с.
9. Борри Х. Firebird: руководство разработчика баз данных. – Санкт-Петербург: БХВ-Петербург, 2006. – 1104 с.
10. Макнамара Р. Документация как код: современные подходы к созданию технических текстов. – Москва: Техносфера, 2021. – 220 с.
11. Relational Database Market Research Report // Market Research Future. – URL: <https://www.marketresearchfuture.com/reports/relational-database-market-18851> (дата обращения: 18.04.2026).
12. Rahm E., Bernstein P.A. A survey of approaches to automatic schema matching // The VLDB Journal. – 2001. – Vol. 10, No. 4. – P. 334–350.

13. Curino C., Moon H.J., Zaniolo C. Graceful database schema evolution: the PRISM workbench // Proceedings of the VLDB Endowment. – 2008. – Vol. 1, No. 1. – P. 761–772.
14. Официальная документация Git // Git SCM. – URL: <https://git-scm.com/doc> (дата обращения: 18.04.2026).
15. ISO/IEC 27001:2022 Information security management systems // ISO. – URL: <https://www.iso.org/standard/27001> (дата обращения: 18.04.2026).